



ADVANCED DATABASE SYSTEMS

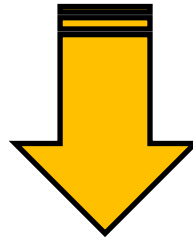
WEEK 7

QUERY OPTIMIZATION PART III



Query Optimization

For a given SQL Query



Goal:

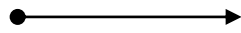
Build the most “optimal” execution plan for the query



Query Optimization

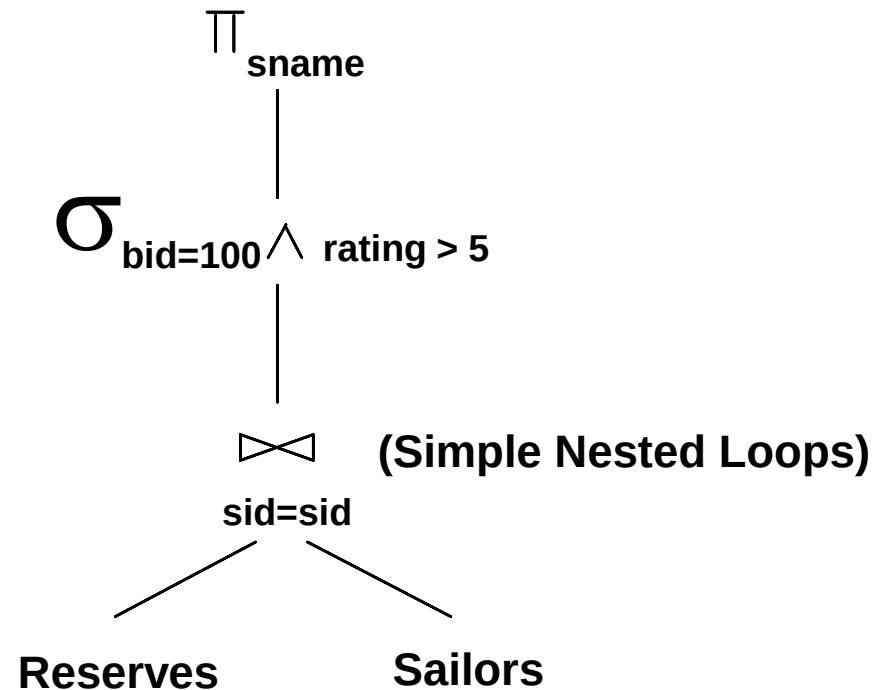
Goal:

Declarative SQL query



Imperative query execution plan:

```
SELECT S.sname
FROM Reserves R, Sailors S
WHERE R.sid = S.sid
      AND R.bid = 100
      AND S.rating > 5
```

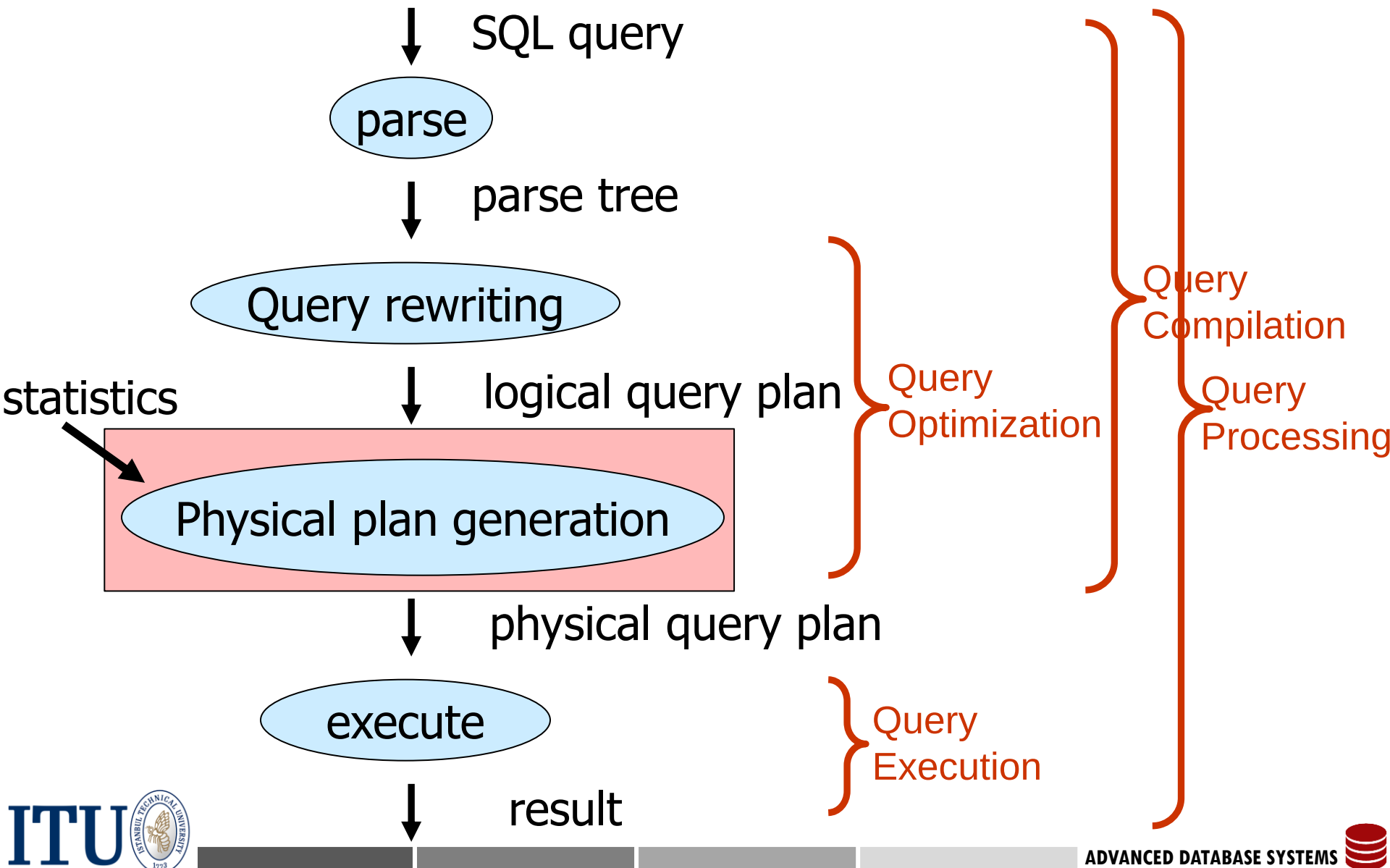


Plan: Tree of R.A. ops, with choice of alg for each op.

Ideally: Want to find best plan. **Practically:** Avoid worst plans!



Overview of Query Processing



Join Ordering Example

- For all relations r_1 , r_2 , and r_3 ,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity)

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.



Join Ordering Example (Cont.)

Consider the expression:

$$\Pi_{name, title}(\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course)$$

- Could compute $teaches \bowtie \Pi_{course_id, title}(course)$ first, and join result with

$$\sigma_{dept_name = \text{“Music”}}(instructor)$$

- However, the result of the first join is likely to be a large relation.
- Only a small fraction of the university's instructors are likely to be from the Music department
 - Better to compute first

$$\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches$$



Cost-Based Optimization

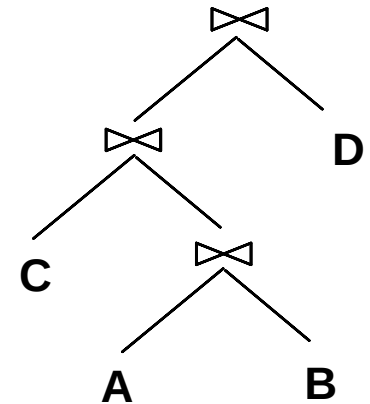
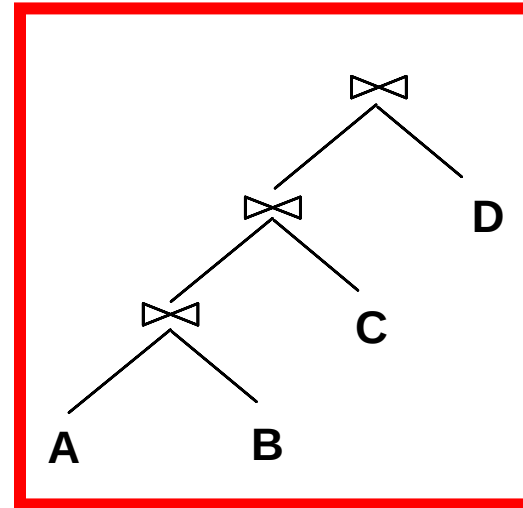
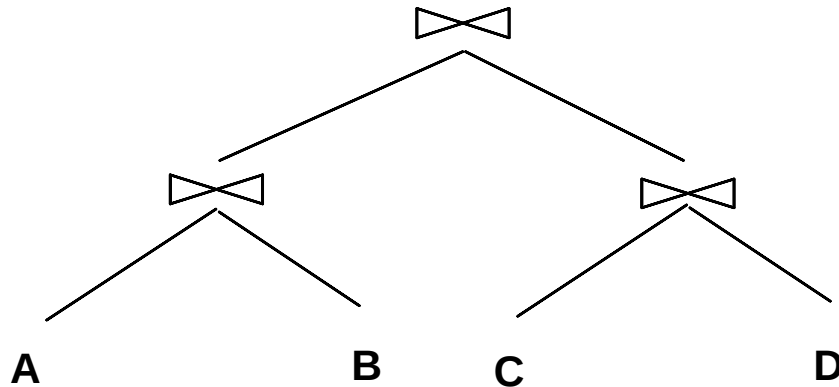
Consider finding the best join-order for $r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$.

- $(2(n - 1))!/(n - 1)!$ different join orders for the above expression.
 - With $n = 7$, the number is 665280,
 - With $n = 10$, the number is greater than 176 billion!
- No need to generate all the join orders.
 - Use dynamic programming!
 - The least-cost join order for any subset of $\{r_1, r_2, \dots, r_n\}$ is computed only once and stored for future use.



Queries Over Multiple Relations

- In principle, need to consider all possible join orderings:



- As the number of joins increases, the number of alternative plans grows rapidly; *we need to restrict the search space.*
- Most RDBMSs consider only left-deep join trees.
 - Left-deep trees allow us to generate **usually** *fully pipelined plans*:
 - ▶ Intermediate results not written to temporary files.
 - Not all left-deep trees are fully pipelined (e.g., SM join).

Cost Functions for the JOIN Operation (cont'd.)

- Multirelation queries and JOIN ordering choices
 - Left-deep join tree
 - Right-deep join tree
 - Bushy join tree

No. of Relations N	No. of Left-Deep Trees $N!$	No. of Bushy Shapes $S(N)$	No. of Bushy Trees $(2N - 2)! / (N - 1)!$
2	2	1	2
3	6	2	12
4	24	5	120
5	120	14	1,680
6	720	42	30,240
7	5,040	132	665,280



Dynamic Programming in Optimization

- To find best join tree for a set S of n relations:
 - Consider all possible plans of the form: $S_1 \bowtie (S - S_1)$ where S_1 is any non-empty subset of S .
 - Recursively compute costs for joining subsets of S to find the cost of each plan.
 - Choose the cheapest of the 2^{n-1} alternatives.
 - Base case for recursion: single relation access plan
 - ▶ Apply all selections on R_i using best choice of indices on R_i
 - When plan for any subset is computed, store it and reuse it when it is required again, instead of recomputing it!
 - ▶ Dynamic programming



Join Order Optimization Algorithm

```
procedure findbestplan(S)
  if (bestplan[S].cost  $\neq \infty$ )
    return bestplan[S]
  // else bestplan[S] has not been computed earlier, compute it now
  if (S contains only 1 relation)
    set bestplan[S].plan and bestplan[S].cost based on the best way
    of accessing S using selections on S and indices (if any) on S
  else
    for each non-empty subset S1 of S such that S1  $\neq$  S
      P1= findbestplan(S1)
      P2= findbestplan(S - S1)
      for each algorithm A for joining results of P1 and P2
        ... compute plan and cost of using A (see next slide) ..
      if cost < bestplan[S].cost
        bestplan[S].cost = cost
        bestplan[S].plan = plan;
    return bestplan[S]
```

Join Order Optimization Algorithm (cont.)

```
for each algorithm  $A$  for joining results of  $P1$  and  $P2$ 
  // For indexed-nested loops join, the outer could be  $P1$  or  $P2$ 
  // Similarly for hash-join, the build relation could be  $P1$  or  $P2$ 
  // We assume the alternatives are considered as separate algorithms
  if algorithm  $A$  is indexed nested loops
    Let  $P_i$  and  $P_o$  denote inner and outer inputs
    if  $P_i$  has a single relation  $r_i$  and  $r_i$  has an index on the join attribute
       $plan =$  “execute  $P_o.plan$ ; join results of  $P_o$  and  $r_i$  using  $A$ ”,
      with any selection conditions on  $P_i$  performed as part of
      the join condition
       $cost = P_o.cost + \text{cost of } A$ 
    else  $cost = \infty$ ; /* cannot use indexed nested loops join */
  else
     $plan =$  “execute  $P1.plan$ ; execute  $P2.plan$ ;
    join results of  $P1$  and  $P2$  using  $A$ .”
     $cost = P1.cost + P2.cost + \text{cost of } A$ 
```

... See previous slide



Cost of Optimization

- With dynamic programming time complexity of optimization with bushy trees is $O(3^n)$.
 - With $n = 10$, this number is 59000 instead of 176 billion!
- Space complexity is $O(2^n)$
- To find best left-deep join tree for a set of n relations:
 - Consider n alternatives with one relation as right-hand side input and the other relations as left-hand side input.
 - Modify optimization algorithm:
 - ▶ replace: “**for each** non-empty subset $S1$ of S such that $S1 \neq S$ ”
 - ▶ by: **for each** relation r in S
let $S1 = S - r$
- If only left-deep trees are considered, time complexity of finding best join order is $O(n 2^n)$
 - Space complexity remains at $O(2^n)$
- Cost-based optimization is expensive, but worthwhile for queries on large datasets (typical queries have small n , generally < 10)



Enumeration of Left-Deep Plans

- Enumerated using N passes (if N relations joined):
 - **Pass 1:** Find best 1-relation plan for each relation.
 - **Pass 2:** Find best way to join result of each 1-relation plan (as outer) to another relation. (*All 2-relation plans.*)
 - **Pass N:** Find best way to join result of a (N-1)-relation plan (as outer) to the N'th relation. (*All N-relation plans.*)
- An N-1 way plan is not combined with an additional relation unless there is a join condition between them, unless all predicates in WHERE have been used up.
 - i.e., **avoid Cartesian products if possible.**
- For each subset of relations, retain only:
 - Cheapest plan overall, plus
 - Cheapest plan for each *interesting order* of the tuples.



Interesting Sort Orders

- Consider the expression $(r_1 \bowtie r_2) \bowtie r_3$ (with A as common attribute)
- An **interesting sort order** is a particular sort order of tuples that could make a later operation (join/group by/order by) cheaper
 - Using merge-join to compute $r_1 \bowtie r_2$ may be costlier than hash join but generates result sorted on A
 - Which in turn may make merge-join with r_3 cheaper, which may reduce cost of join with r_3 and minimizing overall cost
- Not sufficient to find the best join order for each subset of the set of n given relations
 - Must find the best join order for each subset, **for each interesting sort order**
 - Simple extension of earlier dynamic programming algorithms
 - Usually, number of interesting orders is quite small and doesn't affect time/space complexity significantly



Enumeration of Plans (Contd.)

- ORDER BY, GROUP BY, aggregates etc. handled as a final step, using either an 'interestingly ordered' plan or an additional sorting operator.



Sailors:

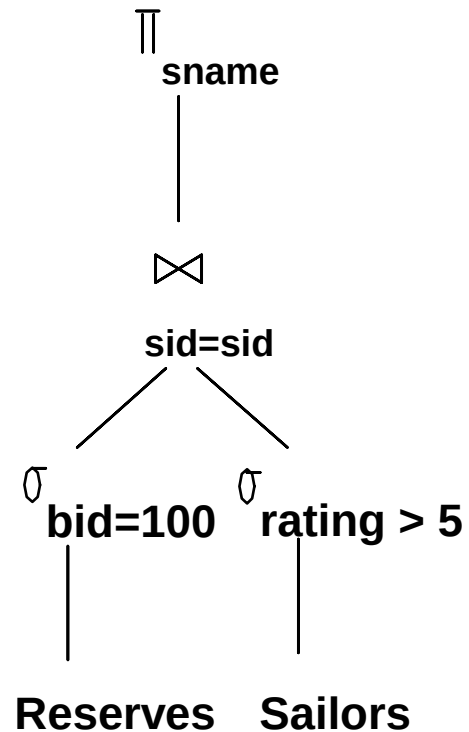
B+ tree on *rating*

Hash on *sid*

Reserves:

B+ tree on *bid*

Example



Pass 1:

- Sailors:** B+ tree matches *rating* > 5, and is probably cheapest. However, if this selection is expected to retrieve a lot of tuples, and index is unclustered, file scan may be cheaper.

- Still, B+ tree plan kept (tuples are in *rating* order).

- Reserves:** B+ tree on *bid* matches *bid*=100; cheapest.

Pass 2: how to join relations?

- e.g., **Reserves as outer:** Hash join could be used to get *Sailors* tuples that satisfy *sid* = outer tuple's *sid* value.



Heuristic Optimization

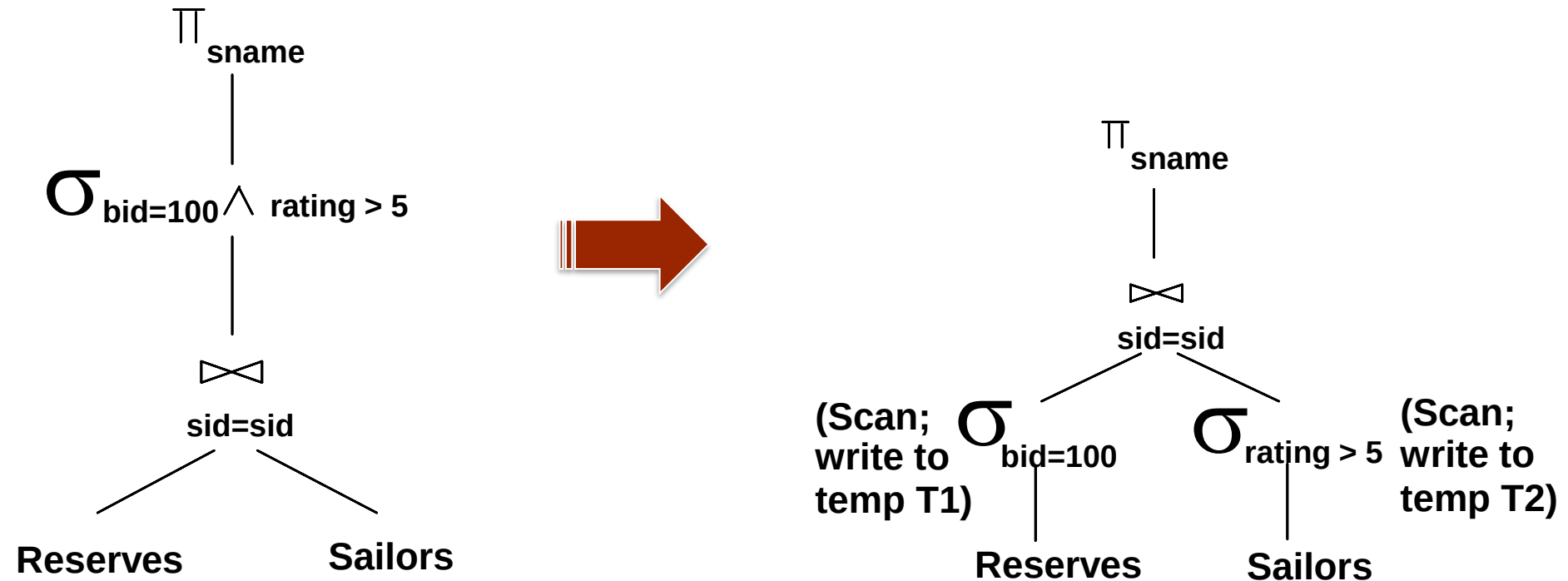
- ❑ Cost-based optimization is expensive, even with dynamic programming.
- ❑ Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.
- ❑ Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - ❑ Perform selection early (reduces the number of tuples)
 - ❑ Perform projection early (reduces the number of attributes)
 - ❑ Perform most restrictive selection and join operations (i.e., with smallest result size) before other similar operations.
- ❑ Some systems use only heuristics, others combine heuristics with partial cost-based optimization.



HEURISTICS FOR ALGEBRAIC OPTIMIZATION



Query Rewriting: Predicate Pushdown



The earlier we process selections, less tuples we need to manipulate higher up in the tree.

Query Rewrites: Predicate Pushdown (more complicated)

```
Select  bid, Max(age)
From    Reserves R, Sailors S
Where   R.sid=S.sid
GroupBy bid
Having  Max(age) > 40
```



```
Select  bid, Max(age)
From    Reserves R, Sailors S
Where   R.sid=S.sid and
        S.age > 40
GroupBy bid
Having  Max(age) > 40
```

- Advantage: the size of the join will be smaller.
- Requires transformation rules specific to the grouping/aggregation operators.
- Will it work if we replace Max by Min?



Query Rewrite: Predicate Movearound

Sailing wizz dates: when did the youngest of each sailor level rent boats?

```
Select  sid, date
From    V1, V2
Where   V1.rating = V2.rating and
        V1.age = V2.age
```

Create View V1 AS
Select rating, Min(age)
From Sailors S
Where S.age < 20
GroupBy rating

Create View V2 AS
Select sid, rating, age, date
From Sailors S, Reserves R
Where R.sid=S.sid

Sailors (sid: integer, sname: string, rating: integer, age: real)
Reserves (sid: integer, bid: integer, day: dates, rname: string)



Query Rewrite: Predicate Movearound

Sailing wizz dates: when did the youngest of each sailor level rent boats?

*First, move
predicates up the
tree.*

```
Select  sid, date
From    V1, V2
Where   V1.rating = V2.rating and
        V1.age = V2.age and age < 20
```

Create View V1 AS
Select rating, Min(age)
From Sailors S
Where S.age < 20
GroupBy rating

Create View V2 AS
Select sid, rating, age, date
From Sailors S, Reserves R
Where R.sid=S.sid

Sailors (sid: integer, sname: string, rating: integer, age: real)
Reserves (sid: integer, bid: integer, day: dates, rname: string)



Query Rewrite: Predicate Movearound

Sailing wizz dates: when did the youngest of each sailor level rent boats?

*First, move
predicates up the
tree.*

```
Select  sid, date
From    V1, V2
Where   V1.rating = V2.rating and
        V1.age = V2.age
```

*Then, move them
down.*

Create View V1 AS
Select rating, Min(age)
From Sailors S
Where S.age < 20
GroupBy rating

Create View V2 AS
Select sid, rating, age, date
From Sailors S, Reserves R
Where R.sid=S.sid and
 S.age < 20

Sailors (sid: integer, sname: string, rating: integer, age: real)
Reserves (sid: integer, bid: integer, day: dates, rname: string)



Structure of Query Optimizers

- Some query optimizers integrate heuristic selection and the generation of alternative access plans.
 - Frequently used approach
 - ▶ heuristic rewriting of nested block structure and aggregation
 - ▶ followed by cost-based join-order optimization for each block
 - Some optimizers (e.g. SQL Server) apply transformations to entire query and do not depend on block structure
- **Optimization cost budget** to stop optimization early (if cost of plan is less than cost of optimization)
- **Plan caching** to reuse previously computed plan if query is resubmitted
 - ▶ Even with different constants in query
- Even with the use of heuristics, cost-based query optimization imposes a substantial overhead.
 - But is worth it for expensive queries
 - Optimizers often use simple heuristics for very cheap queries, and perform exhaustive enumeration for more expensive queries



Statistics for Cost Estimation



Cost Estimation

- For each plan considered, must estimate cost:
 - Must **estimate the cost** of each operation in plan tree.
 - ▶ Depends on input cardinalities.
 - Must **estimate the size of result** for each operation in tree!
 - ▶ Use information about the input relations.
 - ▶ For selections and joins, assume independence of predicates.
- We'll discuss the **System R** cost estimation approach.
 - Very inexact, but works ok in practice.
 - More sophisticated techniques known now.



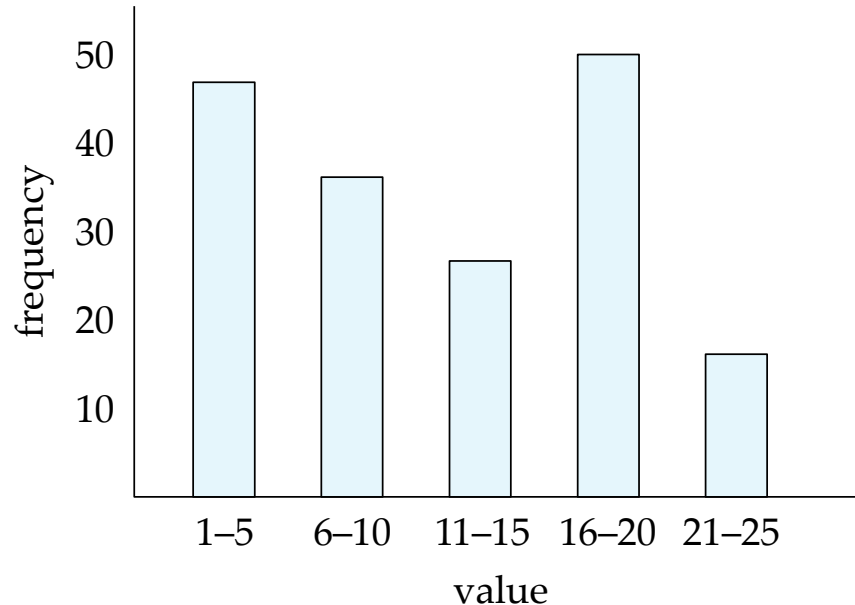
Statistics and Catalogs

- Need information about the relations and indexes involved.
- **Catalogs** typically contain at least:
 - # tuples (NTuples) and # pages (NPages) for each relation.
 - # distinct key values (NKeys) and NPages for each index.
 - Index height, low/high key values (Low/High) for each tree index.
- More detailed information (e.g., histograms of the values in some field) are sometimes stored.



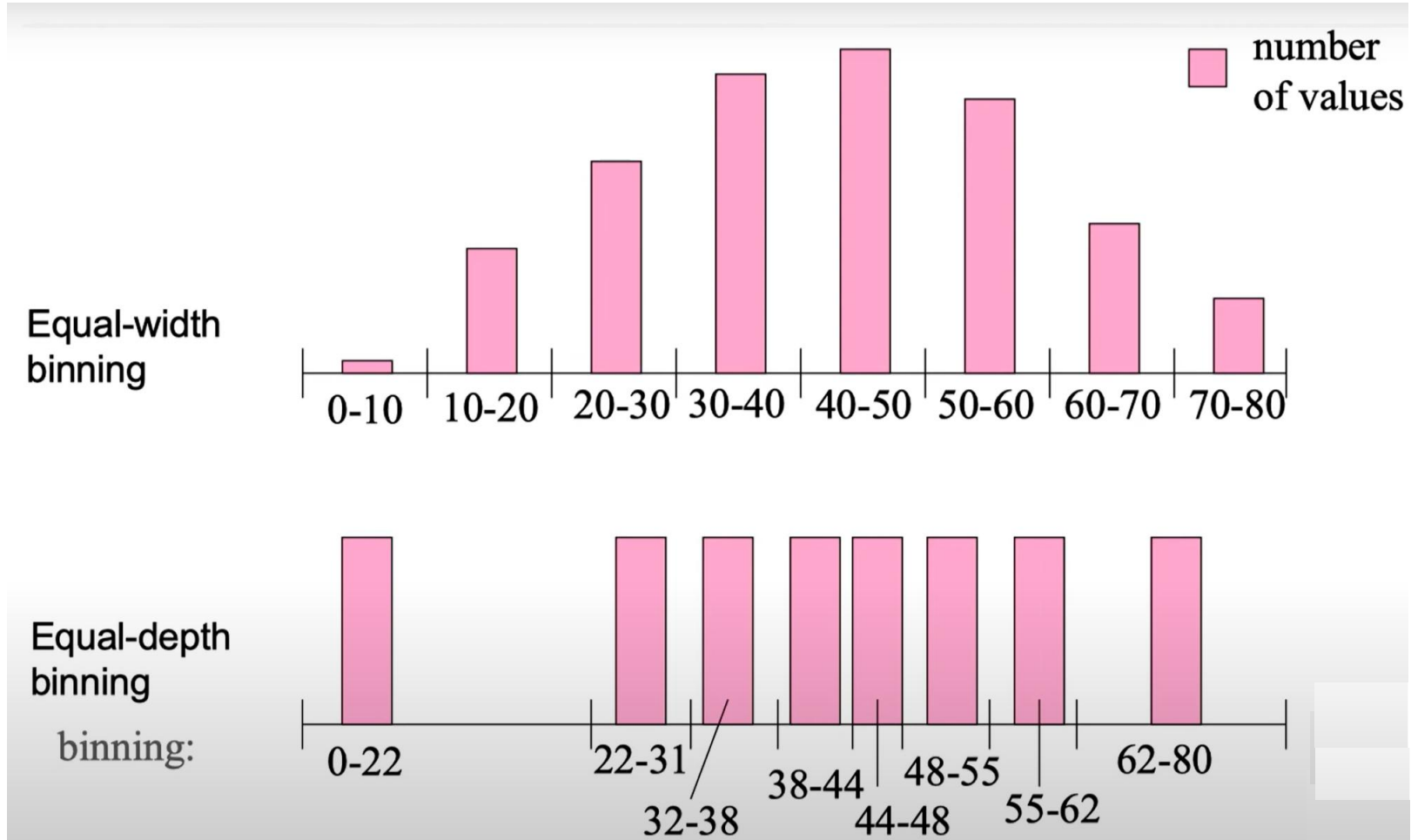
Histograms

- Histogram on attribute *age* of relation *person*



- **Equi-width** histograms
- **Equi-depth** histograms break up range such that each range has (approximately) the same number of tuples
- Many databases also store n **most-frequent values** and their counts
 - Histogram is built on remaining values only

Equi-width vs. Equi-depth Histograms



Computing Statistics

- Histograms and other statistics usually computed based on a **random small sample**
- Statistics may be out of date
 - Catalogs updated periodically.
 - Updating whenever data changes is too expensive; lots of approximation anyway, so slight inconsistency ok.
 - Some databases require an **analyze** command to be executed to update statistics
 - Others automatically recompute statistics
 - ▶ e.g., when number of tuples in a relation changes by some percentage



Cost Functions for SELECT Operation

□ Notation used in cost formulas

C_{S_i} : Cost for method S_i in block accesses

r_X : Number of records (tuples) in a relation X

b_X : Number of blocks occupied by relation X (also referred to as b)

bfr_X : Blocking factor (i.e., number of records per block) in relation X

sl_A : Selectivity of an attribute A for a given condition

s_A : Selection cardinality of the attribute being selected ($= sl_A * r$)

xA : Number of levels of the index for attribute A

$b_{11}A$: Number of first-level blocks of the index on attribute A

$NDV(A, X)$: Number of distinct values of attribute A in relation X



Cost Function for SELECT Operation (cont'd.)

- S1: Linear search (brute force approach)

- Search all file blocks to retrieve all records

$$C_{S1a}=b$$

- For equality condition on a key attribute, on average one-half the records are searched

$$C_{S1b}=\frac{b}{2}$$

- S2: If the data is sorted → Binary search

$$C_{S2}=\log_2 b + \left\lceil \frac{s}{bfr} \right\rceil - 1$$

- ▶ Reduces to $\log_2 b$ if equality condition is on a key attribute

s: selection cardinality
bfr: block factor

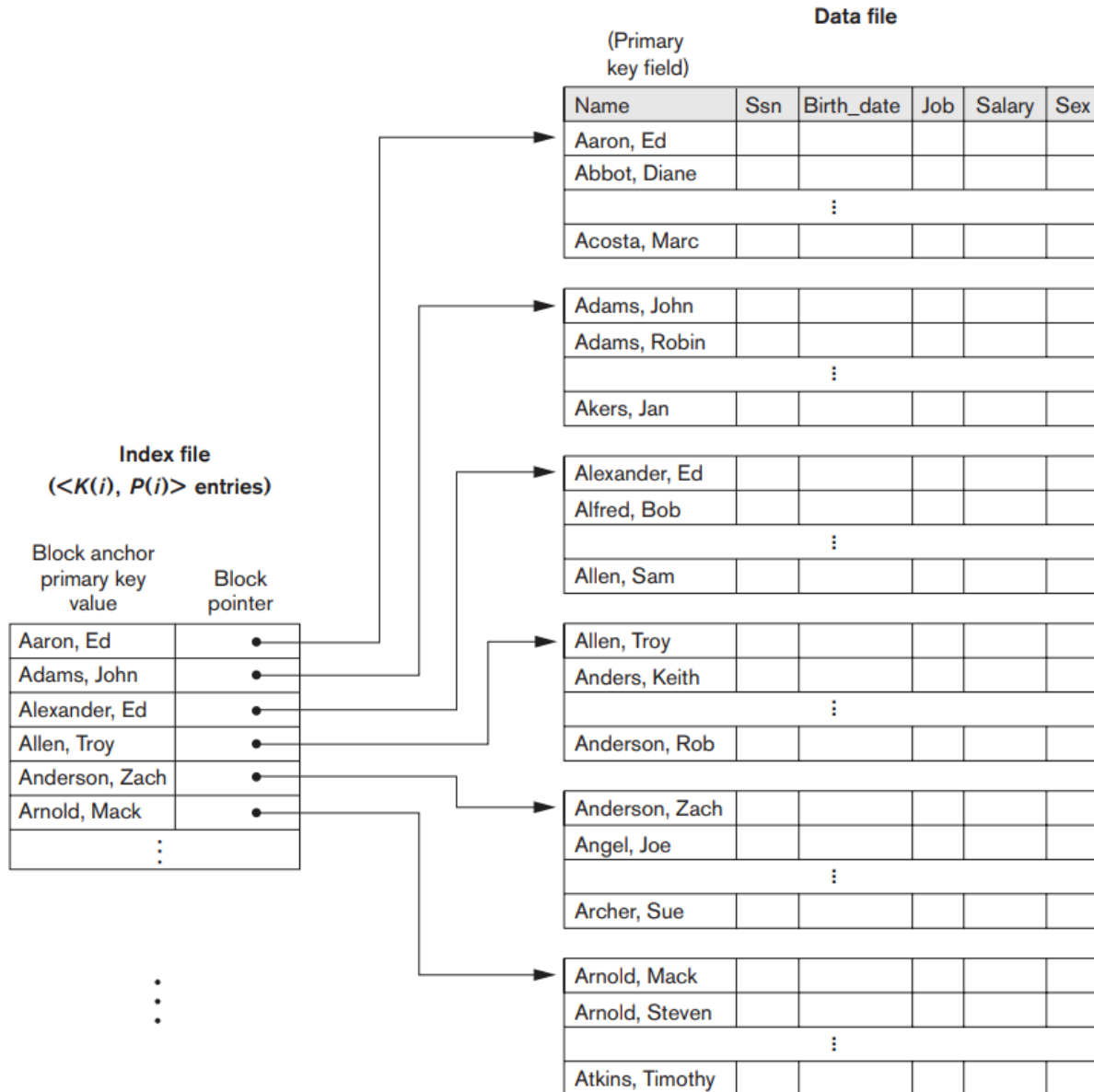


Cost Function for SELECT Operation (cont'd.)

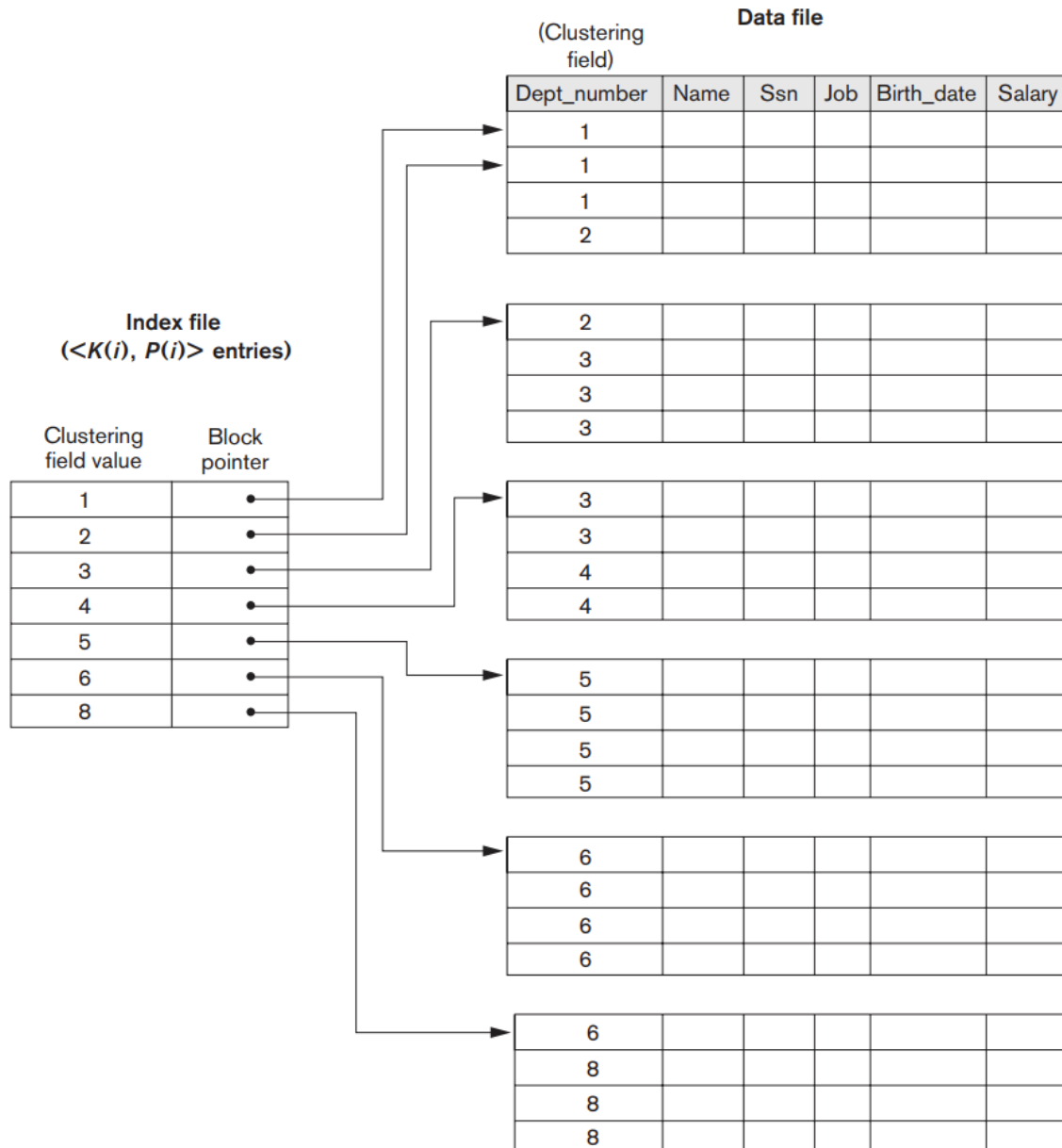
- S3: Using a primary index (on a key column) to retrieve a single record
 - S3a: Tree-based index
 - ▶ $C_{S3a} = x + 1$
 - S3b: Using a hash key to retrieve a single record
 - ▶ $C_{S3b} = 1 + 1$
- S4: Using a clustered index (on a non-key column) to retrieve multiple records with range queries ($>$, $>=$, $<$, $<=$)
 - $C_{S4} = x + \frac{b}{2}$



Primary Index



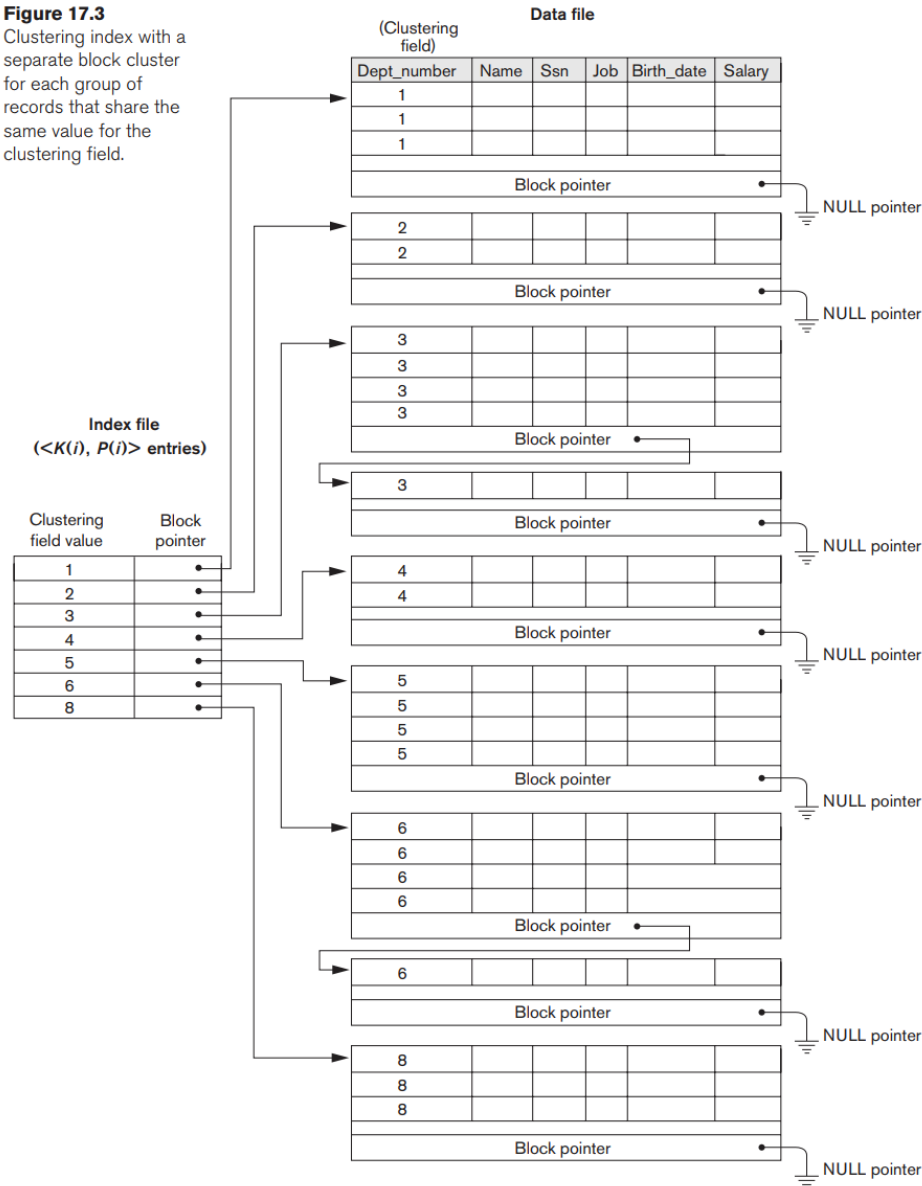
Clustering Index



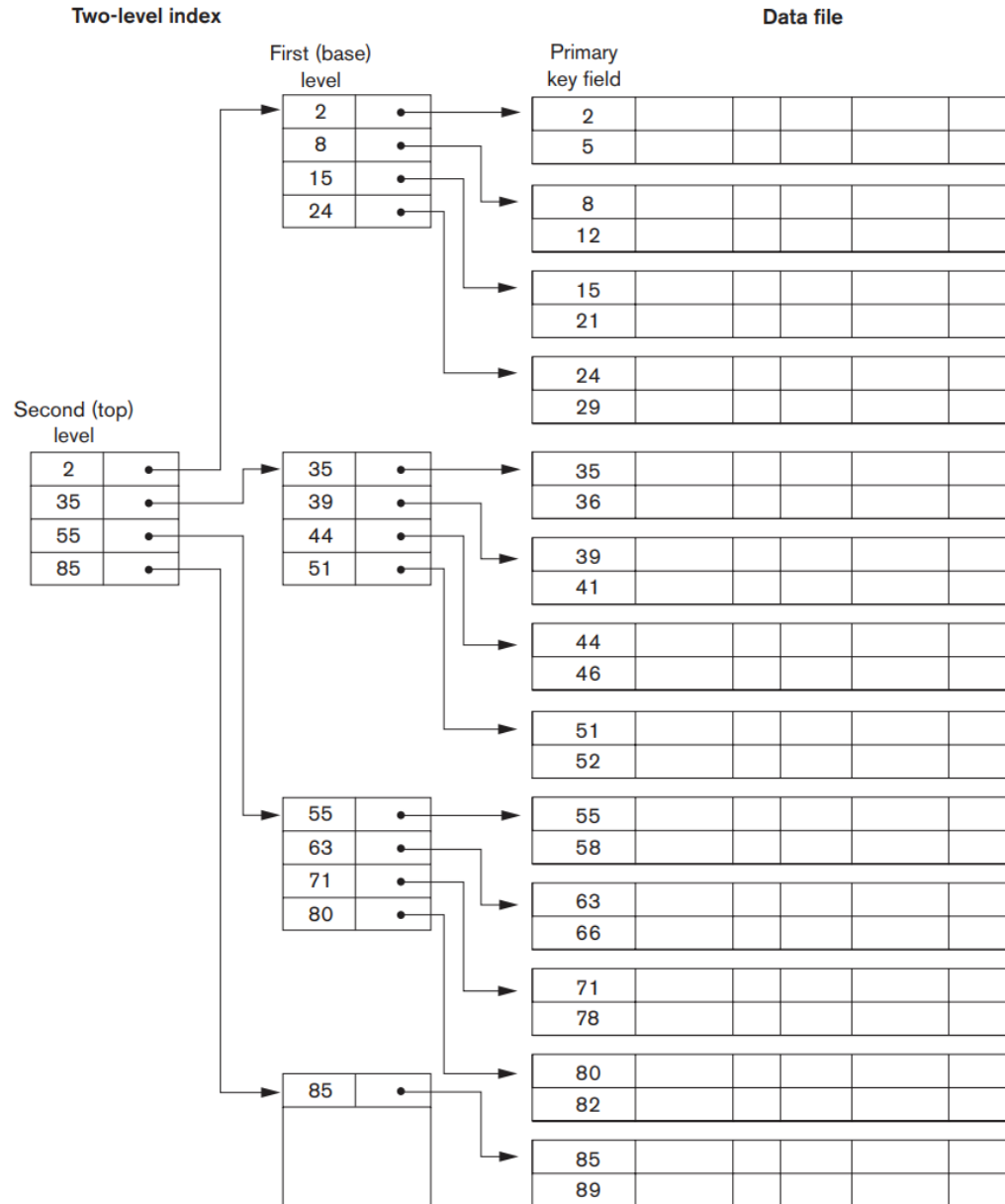
Clustering index with a separate block cluster

Figure 17.3

Clustering index with a separate block cluster for each group of records that share the same value for the clustering field.

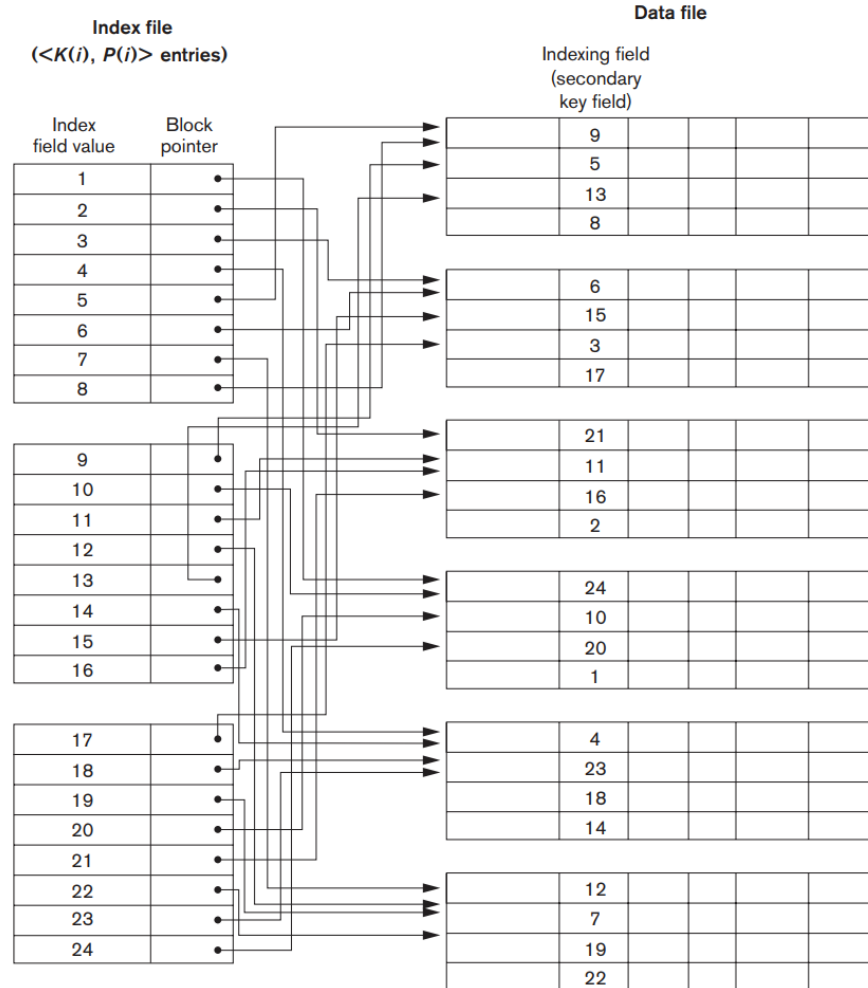


Multi-level Primary Index

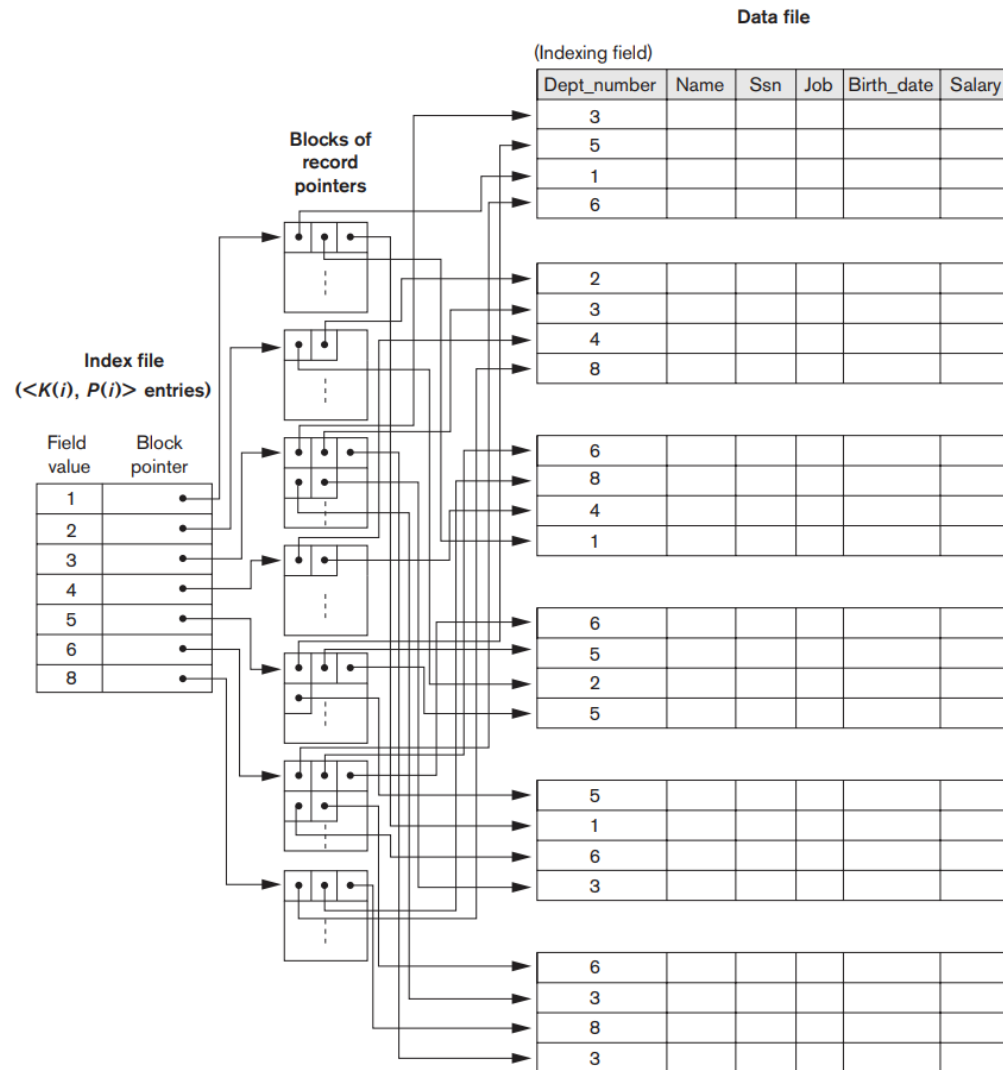


Secondary Index with Block Pointers

A dense secondary index (with block pointers) on a nonordering key field of a file.



Extra Level of Indirection for Indexes on Non-key Values



Cost Function for SELECT Operation (cont'd.)

- S5 - Using a secondary clustered B+-tree index to retrieve multiple records.

- $C_{S5a} = x + 1 + \left\lceil \frac{s}{bfr} \right\rceil$ (equality condition)

- $C_{S5b} = x + \frac{bI1}{2} + \frac{b}{2}$ (range queries)

- S6 - Using a secondary unclustered B+-tree index

- $C_{S6a} = x + 1 + s$ (equality condition)

- $C_{S6b} = x + \frac{bI1}{2} + \frac{r}{2}$ (range queries)



Size Estimation of Complex Selections

- The **selectivity** of a condition θ_i is the probability that a tuple in the relation r satisfies θ_i .
 - If s_i is the number of satisfying tuples in R , the selectivity of θ_i is given by $s_i / |R|$.
- **Conjunction:** $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(R)$. Assuming independence, the estimated number of tuples in the result is:

$$|R| * \frac{s_1 * s_2 * \dots * s_n}{|R|^n}$$

- **Disjunction:** $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$. Estimated number of tuples:

$$|R| * \left(1 - \left(1 - \frac{s_1}{|R|} \right) * \left(1 - \frac{s_2}{|R|} \right) * \dots * \left(1 - \frac{s_n}{|R|} \right) \right)$$

- **Negation:** $\sigma_{\neg \theta}(R)$. Estimated number of tuples:

$$|R| - \text{size}(\sigma_{\theta}(R)) = |R| * \left(1 - \frac{s_{\theta}}{|R|} \right)$$



Cost Functions for the JOIN Operation

- Cost functions involve the estimation of file size that results from the JOIN operation
- Join selectivity
 - Ratio of the size of resulting file to size of the CARTESIAN PRODUCT file
 - Simple formula for join selectivity

$$js = |(R \bowtie_c S)| / |(R \times S)| = |(R \bowtie_c S)| / (|R| * |S|)$$

- Join cardinality

$$js = 1 / \max (\text{NDV} (A, R), \text{NDV} (B, S))$$

$$jc = |(R_c S)| = js * |R| * |S|$$



Estimation of the Size of Joins

- If $\text{Schema}(R) \cap \text{Schema}(S) = \{A\}$ is not a key for R or S .
If we assume that every tuple t in R produces tuples in $R \bowtie S$, the number of tuples in $R \bowtie S$ is estimated to be:

$$|R| * \frac{|S|}{NDV(A, S)}$$

If the reverse is true, the estimate obtained will be:

$$|S| * \frac{|R|}{NDV(A, R)}$$

The lower of these two estimates is probably the more accurate one.

$$\frac{|R| * |S|}{\max(NDV(A, R), NDV(A, S))}$$

- Can improve on above if histograms are available
 - Use formula similar to above, for each cell of histograms on the two relations



Estimation of the Size of Joins (Special Cases)

- The Cartesian product $R \times S$ contains $|R| * |S|$ tuples; each tuple occupies $s_r + s_s$ bytes.
- If $\text{Schema}(R) \cap \text{Schema}(S) = \emptyset$, then $R \bowtie S$ is the same as $R \times S$.
- If $\text{Schema}(R) \cap \text{Schema}(S)$ is a key for R , then a tuple of S will join with at most one tuple from R
 - therefore, the number of tuples in $R \bowtie S$ is no greater than the number of tuples in S .
- If $\text{Schema}(R) \cap \text{Schema}(S)$ is a foreign key in S referencing R , then the number of tuples in $r \bowtie s$ is exactly the same as the number of tuples in S .
 - ▶ The case for $\text{Schema}(R) \cap \text{Schema}(S)$ being a foreign key in R referencing S is symmetric.



Estimation of the Size of Joins – General Case

```
SELECT attribute list  
FROM relation list  
WHERE term1 AND ... AND termk
```

- Consider a query block:
- Maximum # tuples in result is the product of the cardinalities of relations in the FROM clause.
- *Selectivity* associated with each *term* reflects the impact of the *term* in reducing result size.
- *Result cardinality* = Max # tuples * product of all selectivities.
 - Implicit *assumption* that *terms* are independent!
 - Term $col=value$ has selectivity $1/NDV(col)$
 - Term $col1=col2$ has selectivity $1/MAX(NDV(col1), NDV(col2))$
 - Term $col>value$ has selectivity $(High(col)-value)/(High(col)-Low(col))$



Cost Functions for the JOIN Operation (cont'd.)

- J1: Page/Block-based Nested-loop join

- For three memory buffer blocks:

$$C_{J1} = b_R + (b_R * b_S) + ((js * |R| * |S|)/bfr_{RS})$$

- For n_B memory buffer blocks:

$$C_{J1} = b_R + (\lceil b_R/(n_B - 2) \rceil * b_S) + ((js * |R| * |S|)/bfr_{RS})$$

- J2: Index-based nested-loop join

- For a secondary index with selection cardinality s_B for join attribute B of S:

$$C_{J2a} = b_R + (|R| * (x_B + 1 + s_B)) + ((js * |R| * |S|)/bfr_{RS})$$



Cost Functions for the JOIN Operation (cont'd.)

- J3: Sort-merge join

- For files already sorted on the join attributes

$$C_{J3a} = b_R + b_S + ((js * |R| * |S|)/bfr_{RS})$$

- Cost of sorting must be added if sorting needed

- J4: Partition-hash join

$$C_{J4} = 3 * (b_R + b_S) + ((js * |R| * |S|)/bfr_{RS})$$



Example: Cost-Based Query Optimization

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM   PROJECT, DEPARTMENT, EMPLOYEE
WHERE  Dnum=Dnumber AND Mgr_ssn=Ssn AND
       Plocation='Stafford';
```

(a)

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

(b)

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000

(c)

Index_name	Uniqueness	Blevel*	Leaf_blocks	Distinct_keys
PROJ_PLOC	NONUNIQUE	1	4	200
EMP_SSN	UNIQUE	1	50	10000
EMP_SAL	NONUNIQUE	1	50	500

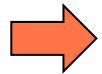
*Blevel is the number of levels without the leaf level.

Sample statistical
information for
relations in the query.
(a) Column statistics
(b) Table statistics
(c) Index statistics



Example: Cost-Based Query Optimization

- Assume optimizer considers only left-deep trees
- Evaluate potential join orders – avoiding cartesian product



- PROJECT ⋈ DEPARTMENT ⋈ EMPLOYEE
- DEPARTMENT ⋈ PROJECT ⋈ EMPLOYEE
- DEPARTMENT ⋈ EMPLOYEE ⋈ PROJECT
- EMPLOYEE ⋈ DEPARTMENT ⋈ PROJECT



Example: Cost-Based Query Optimization

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum=Dnumber AND Mgr_ssn=Ssn AND
Plocation='Stafford';
```

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

- Let's compute the cost of the first join order:

PROJECT ⋈ DEPARTMENT ⋈ EMPLOYEE

- Need to determine:

- Access path to each relation
- Join method for each pair

- Dept:

- No index → Full scan

- Project:

- First do the selection on location. Two options:
 - Full scan or use PROJ_PLOC index
 - PROJ_PLOC index: (unclustered, non-unique as Plocation is not a key)
 - Selectivity: $1/NDV(Plocation)$
 - Cardinality after filtering: $1/200 * 2000 = 10$
 - Estimated cost: 12 block accesses (2 for the ix and 10 for data)

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000

Index_name	Uniqueness	Blevel*	Leaf_blocks	Distinct_keys
PROJ_PLOC	NONUNIQUE	1	4	200
EMP_SSN	UNIQUE	1	50	10000
EMP_SAL	NONUNIQUE	1	50	500

Full Scan Cost: 100 blocks (Index access is cheaper!)



Example: Cost-Based Query Optimization

- Result will be stored in a TEMP1 table with 1 block.
 - blocking factor $2000/100 = 20$ tuples per block $\rightarrow$ 10 rows fit into a single block
- Nested Loops
 - TEMP1 as outer: $1 + 1 * 5$ (Dept table blocks)
 - Writing the result of the join
 - ▶ Estimating the result of the join
 - Dnumber is the key for dept.
 - Any row of Temp1 will match at most 1 tuple from Dept.
 - Therefore join size is at most the size of Temp1.
 - Assume blocking factor of 5
 - 2 blocks will be written in the result.

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM   PROJECT, DEPARTMENT, EMPLOYEE
WHERE  Dnum=Dnumber AND Mgr_ssn=Ssn AND
       Plocation='Stafford';
```

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000



Join Operation: Running Example

Running example:

$student \bowtie takes$

Catalog information:

- $r_{student} = 5,000$.
- $bfr_{student} = 50$, which implies that
 $b_{student} = 5000/50 = 100$.
- $r_{takes} = 10000$.
- $bfr_{takes} = 25$, which implies that
 $b_{takes} = 10000/25 = 400$.
- $NDV(ID, takes) = 2500$, which implies that on average, each student has taken 4 courses.
 - Attribute ID in $takes$ is a foreign key referencing $student$.
 - $NDV(ID, student) = 5000$ (primary key!)



Estimation of the Size of Joins

□ Reminder:

- If $\text{Schema}(R) \cap \text{Schema}(S)$ in S is a foreign key in S referencing R , then the number of tuples in $R \bowtie S$ is exactly the same as the number of tuples in S .
 - ▶ The case for $\text{Schema}(R) \cap \text{Schema}(S)$ being a foreign key referencing S is symmetric.

□ In the example query $\text{student} \bowtie \text{takes}$, ID in takes is a foreign key referencing student

- hence, the result has exactly r_{takes} tuples, which is 10000

Estimation of the Size of Joins

- Compute the size estimates for *student* ⋈ *takes* without using information about foreign keys:
 - $NDV(ID, takes) = 2500$, and $NDV(ID, student) = 5000$
 - The two estimates are $5000 * 10000/2500 = 20,000$ and $5000 * 10000/5000 = 10000$
 - We choose the lower estimate, which in this case, is the same as our earlier computation using foreign keys.



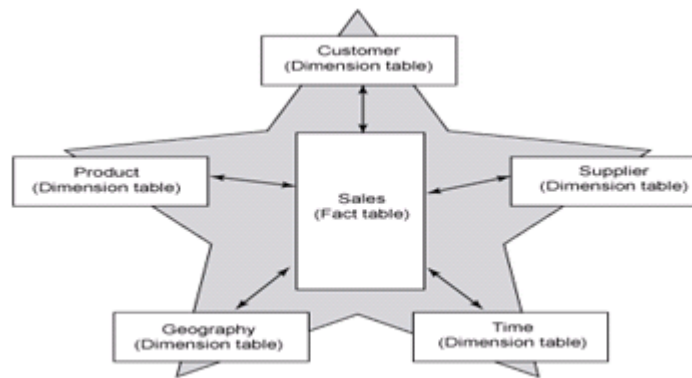
Additional Issues Related to Query Optimization (cont'd.)

- Size estimation of other operations
 - Projection: $|R|$ or $NDV(R)$
 - Set operations:
 - ▶ Union: $|R| + |S|$
 - ▶ Intersection: $\min(|R|, |S|)$
 - Aggregation with Group By
 - ▶ $NDV(C, R)$



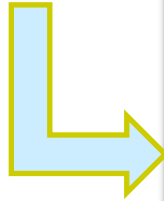
An Example of Query Optimization in Data Warehouses

- Star transformation optimization
 - Goal: access a reduced set of data from the fact table and avoid using a full table scan on it
 - Classic star transformation
 - Bitmap index star transformation
- Joining back



Star Transformation

```
SELECT c.cust_city,  
       t.calendar_quarter_desc,  
       SUM(s.amount_sold) sales_amount  
FROM sales s,  
     times t,  
     customers c,  
     channels ch  
WHERE s.time_id = t.time_id  
AND s.cust_id = c.cust_id  
AND s.channel_id = ch.channel_id  
AND c.cust_state_province = 'CA'  
AND ch.channel_desc = 'Internet'  
AND t.calendar_quarter_desc IN ('1999-01','1999-02')  
GROUP BY c.cust_city, t.calendar_quarter_desc;
```



```
SELECT c.cust_city,  
       t.calendar_quarter_desc,  
       SUM(s.amount_sold) sales_amount  
FROM sales s,  
     times t,  
     customers c  
WHERE s.time_id = t.time_id  
AND s.cust_id = c.cust_id  
AND c.cust_state_province = 'CA'  
AND t.calendar_quarter_desc IN ('1999-01','1999-02')  
AND s.time_id IN (SELECT time_id  
                  FROM times  
                  WHERE calendar_quarter_desc IN('1999-01','1999-02'))  
AND s.cust_id IN (SELECT cust_id  
                  FROM customers  
                  WHERE cust_state_province='CA')  
AND s.channel_id IN (SELECT channel_id  
                     FROM channels  
                     WHERE channel_desc = 'Internet')  
GROUP BY c.cust_city, t.calendar_quarter_desc;
```



Additional Optimization techniques



Optimizing Nested Subqueries**

- Nested query example:

```
select name
from instructor
where exists (select *
               from teaches
               where instructor.ID = teaches.ID and teaches.year = 2019)
```

- SQL conceptually treats nested subqueries in the where clause as functions that take parameters and return a single value or set of values
 - Parameters are variables from outer level query that are used in the nested subquery; such variables are called **correlation variables**
- Conceptually, nested subquery is executed once for each tuple in the cross-product generated by the outer level **from** clause
 - Such evaluation is called **correlated evaluation**
 - Note: other conditions in where clause may be used to compute a join (instead of a cross-product) before executing the nested subquery



Optimizing Nested Subqueries

- ❑ Correlated evaluation may be quite inefficient since
 - ❑ a large number of calls may be made to the nested query
 - ❑ there may be unnecessary random I/O as a result
- ❑ SQL optimizers attempt to transform nested subqueries to joins where possible, enabling use of efficient join techniques

```
select name
from instructor
where exists (select *
               from teaches
               where instructor.ID = teaches.ID
               and teaches.year = 2019)
```



```
select name
from teaches, instructor
where instructor.ID = teaches.ID
       and teaches.year = 2019
```

- ❑ Note: the two queries generate different numbers of duplicates (why?)
 - ❑ Can be modified to handle duplicates correctly using semijoins



Optimizing Nested Subqueries

- The **semijoin** operator $\bowtie$ is defined as follows
 - A tuple r_i appears n times in $r \bowtie_{\theta} s$ if it appears n times in r , and there is at least one matching tuple s_i in s

```
select name
from instructor
where ID in (select teaches.ID
                 from teaches
                 where teaches.year = 2019)
```

- The above SQL query can be written in relational algebra as:

$$\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$$

- Or even as:

$$\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID} (\sigma_{teaches.year=2019} teaches))$$

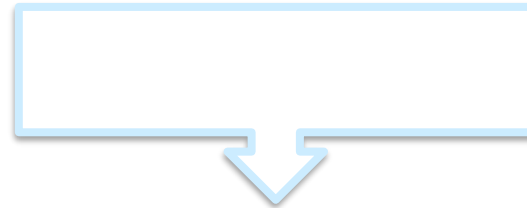
- Now the duplicate count is correct!



Query Rewrite: Unnesting Queries

```
SELECT S.sname  
FROM Sailors S  
WHERE EXISTS  
  (SELECT *  
   FROM Reserves R  
   WHERE R.bid=103  
    AND R.sid=S.sid)
```

Nested block to optimize:
SELECT *
FROM Reserves R
WHERE R.bid=103
AND S.sid= *outer value*



Equivalent non-nested query:
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid=R.sid
AND R.bid=103

□ *The non-nested version of the query is typically optimized better.*



Optimizing Nested Subqueries

- This could also be written using only joins (in SQL) as

```
with  $t_1$  as
  (select distinct ID
   from teaches
   where year = 2019)
select name
from instructor,  $t_1$ 
where  $t_1.ID$  = instructor.ID
```



Optimizing Nested Subqueries

□ The query

```
select name
from instructor
where not exists (select *
                  from teaches
                  where instructor.ID = teaches.ID and teaches.year = 2019)
```

can be rewritten using the **anti-semijoin** operation as $\bar{\bowtie}$

$$\Pi_{name}(instructor \bar{\bowtie}_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$$

The **anti-semijoin** operator $\bar{\bowtie}$:

A tuple r_i appears n times in $r \bar{\bowtie}_{\theta} s$ if it appears n times in r , and there is **no** matching tuple s_j in s

Optimizing Nested Subqueries (Cont.)

In general, SQL queries of the form below can be rewritten as shown

- Rewrite: **select** A
from $r_1, r_2, \dots, r_n$
where P_1 **and exists** (**select** $*$
from $s_1, s_2, \dots, s_m$
where P_2^1 **and** P_2^2)
- To: $\Pi_A(\sigma_{P_1}(r_1 \times r_2 \times \dots \times r_n) \bowtie_{P_2^2} \sigma_{P_2^1}(s_1 \times s_2 \times \dots \times s_m))$
 - P_2^1 contains predicates that do not involve any correlation variables
 - P_2^2 contains predicates involving correlation variables
- The process of replacing a nested query by a query with a join/semijoin (possibly with a temporary relation) is called **decorrelation**.
- Decorrelation is more complicated in several cases, e.g.
 - ▶ The nested subquery uses aggregation, or
 - ▶ The nested subquery is a scalar subquery
- Correlated evaluation used in these cases

Outer Join Size Estimation

□ R FULL OUTER JOIN S:

▶ $|R \text{ join } S| + |R \text{ anti-join } S| + |S \text{ anti-join } R|$



Cost Functions for the JOIN Operation (cont'd.)

- Join selectivity and cardinality for semi-join

```
SELECT COUNT(*)  
FROM T1  
WHERE T1.X IN (SELECT T2.Y  
               FROM T2);
```

- Unnesting query above leads to semi-join

```
SELECT COUNT(*)  
FROM T1, T2  
WHERE T1.X S= T2.Y;
```

- Join selectivity

$$js = \text{MIN}(1, \text{NDV}(Y, T2) / \text{NDV}(X, T1))$$

- Join cardinality

$$jc = |T1|^* js$$



Cost Functions for the JOIN Operation (cont'd.)

- Join selectivity and cardinality for anti-join

```
SELECT COUNT (*)  
FROM T1  
WHERE T1.X NOT IN (SELECT T2.Y  
FROM T2);
```

- Unnesting query above leads to anti-join

```
SELECT COUNT(*)  
FROM T1, T2  
WHERE T1.X A= T2.Y;
```

- Join selectivity

$$js = 1 - \text{MIN}(1, \text{NDV}(T2.y) / \text{NDV}(T1.x))$$

- Join cardinality

$$jc = |T1| * js$$



Materialized Views

- A **materialized view** is a view whose contents are computed and stored.
- Consider the view

```
create view department_total_salary(dept_name, total_salary) as  
select dept_name, sum(salary)  
from instructor  
group by dept_name
```

- Materializing the above view would be very useful if the total salary by department is required frequently
 - Saves the effort of finding multiple tuples and adding up their amounts



Materialized View Maintenance

- The task of keeping a materialized view up-to-date with the underlying data is known as **materialized view maintenance**
- Materialized views can be maintained by recomputation on every update
- A better option is to use **incremental view maintenance**
 - **Changes to database relations are used to compute changes to the materialized view, which is then updated**
- View maintenance can be done by
 - Manually defining triggers on insert, delete, and update of each relation in the view definition
 - Manually written code to update the view whenever database relations are updated
 - Periodic recomputation (e.g. nightly)
 - Incremental maintenance supported by many database systems
 - ▶ Avoids manual effort/correctness issues



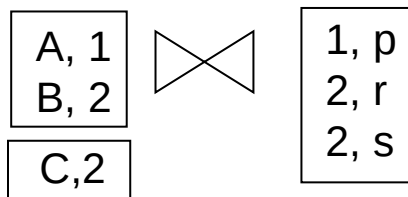
Incremental View Maintenance

- The changes (inserts and deletes) to a relation or expressions are referred to as its **differential**
 - Set of tuples inserted to and deleted from r are denoted i_r and d_r
- To simplify our description, we only consider inserts and deletes
 - We replace updates to a tuple by deletion of the tuple followed by insertion of the update tuple
- We describe how to compute the change to the result of each relational operation, given changes to its inputs
- We then outline how to handle relational algebra expressions



Join Operation

- Consider the materialized view $v = r \bowtie s$ and an update to r
- Let r^{old} and r^{new} denote the old and new states of relation r
- Consider the case of an insert to r :
 - We can write $r^{new} \bowtie s$ as $(r^{old} \cup i_r) \bowtie s$
 - And rewrite the above to $(r^{old} \bowtie s) \cup (i_r \bowtie s)$
 - But $(r^{old} \bowtie s)$ is simply the old value of the materialized view, so the incremental change to the view is just $i_r \bowtie s$
- Thus, for inserts $v^{new} = v^{old} \cup (i_r \bowtie s)$
- Similarly for deletes $v^{new} = v^{old} - (d_r \bowtie s)$



A, 1, p
B, 2, r
B, 2, s
C, 2, r
C, 2, s



Selection and Projection Operations

- Selection: Consider a view $v = \sigma_{\theta}(r)$.
 - $v^{new} = v^{old} \cup \sigma_{\theta}(i_r)$
 - $v^{new} = v^{old} - \sigma_{\theta}(d_r)$
- Projection is a more difficult operation
 - $R = (A,B)$, and $r = \{ (a,2), (a,3) \}$
 - $\Pi_A(r)$ has a single tuple (a) .
 - If we delete the tuple $(a,2)$ from r , we should not delete the tuple (a) from $\Pi_A(r)$, but if we then delete $(a,3)$ as well, we should delete the tuple
- For each tuple in a projection $\Pi_A(r)$, we will keep a count of how many times it was derived
 - On insert of a tuple to r , if the resultant tuple is already in $\Pi_A(r)$ we increment its count, else we add a new tuple with count = 1
 - On delete of a tuple from r , we decrement the count of the corresponding tuple in $\Pi_A(r)$
 - ▶ if the count becomes 0, we delete the tuple from $\Pi_A(r)$



Aggregation Operations

□ **Count** : $v = \gamma_{count(B)}^{(r)}.$

□ When a set of tuples i_r is inserted

- ▶ For each tuple r in i_r , if the corresponding group, $r.A$, is already present in v , we increment its count, else we add a new tuple with count = 1

□ When a set of tuples d_r is deleted

- ▶ for each tuple t in d_r , we look for the group $t.A$ in v , and subtract 1 from the count for the group.
 - If the count becomes 0, we delete from v the tuple for the group $t.A$

□ **Sum**: $v = \gamma_{sum(B)}^{(r)}$

- We maintain the sum in a manner similar to count, except we add/subtract the B value instead of adding/subtracting 1 for the count
- Additionally we maintain the count in order to detect groups with no tuples. Such groups are deleted from v
 - ▶ Cannot simply test for sum = 0 (why?)



Aggregate Operations (Cont.)

- **Avg**: How to handle average?
 - Maintain **sum** and **count** separately, and divide at the end
- **min, max**: $V = \gamma_{\min(B)}(r)$.
 - Handling insertions on r is straightforward.
 - Maintaining the aggregate values **min** and **max** on deletions may be more expensive. We have to look at the other tuples of r that are in the same group to find the new minimum



Other Operations

- Set intersection: $v = r \cap s$
 - When a tuple is inserted in r , we check if it is present in s , and if so we add it to v . If it already exists, increment count by 1.
 - If the tuple is deleted from r , decrement the count by 1. We delete it from the intersection if it is present there and count=0.
 - Updates to s are symmetric
 - The other set operations, *union* and *set difference* are handled in a similar fashion.
- Outer joins are handled in much the same way as joins but with some extra work



Handling Expressions

- To handle an entire expression, we derive expressions for computing the incremental change to the result of each sub-expressions, starting from the smallest sub-expressions.
- E.g., consider $E_1 \bowtie E_2$ where each of E_1 and E_2 may be a complex expression
 - Suppose the set of tuples to be inserted into E_1 is given by D_1
 - ▶ Computed earlier, since smaller sub-expressions are handled first
 - Then the set of tuples to be inserted into $E_1 \bowtie E_2$ is given by $D_1 \bowtie E_2$
 - ▶ This is just the usual way of maintaining joins



Query Optimization and Materialized Views

- Rewriting queries to use materialized views:
 - A materialized view $v = r \bowtie s$ is available
 - A user submits a query $r \bowtie s \bowtie t$
 - We can rewrite the query as $v \bowtie t$
 - ▶ Whether to do so depends on cost estimates for the two alternative
- Replacing a use of a materialized view by the view definition:
 - A materialized view $v = r \bowtie s$ is available, but without any index on it
 - User submits a query $\sigma_{A=10}(v)$.
 - Suppose also that s has an index on the common attribute B , and r has an index on attribute A .
 - The best plan for this query may be to replace v by $r \bowtie s$, which can lead to the query plan $\sigma_{A=10}(r) \bowtie s$
- Query optimizer should be extended to consider all above alternatives and choose the best overall plan



Materialized View Selection

- **Materialized view selection:** “What is the best set of views to materialize?”
- **Index selection:** “what is the best set of indices to create”
 - closely related to materialized view selection
 - ▶ but simpler
- Materialized view selection and index selection based on typical system **workload** (queries and updates)
 - Typical goal: minimize time to execute workload, subject to constraints on space and time taken for some critical queries/updates
 - One of the steps in database tuning
- Commercial database systems provide tools (called “tuning assistants” or “wizards”) to help the database administrator choose what indices and materialized views to create



Top-K Queries

□ Top-K queries

```
select *  
from r, s  
where r.B = s.B  
order by r.A ascending  
limit 10
```

- Alternative 1: Indexed nested loops join with r as outer
- Alternative 2: estimate highest r.A value in result and add selection (**and** r.A $\leq$ H) to where clause
 - ▶ If < 10 results, retry with larger H



Join Minimization

□ Join minimization

```
select r.A, r.B  
from r, s  
where r.B = s.B
```

□ Check if join with s is redundant, drop it

- E.g., join condition is on foreign key from r to s, r.B is declared as not null, and no selection on s

□ Other sufficient conditions possible

```
select r.A, s2.B  
from r, s as s1, s as s2  
where r.B=s1.B and r.B = s2.B and s1.A < 20 and s2.A < 10
```

- ▶ join with s1 is redundant and can be dropped (along with selection on s1)

□ Lots of research in this area since 70s/80s!



Multiquery Optimization

□ Example

Q1: **select * from (r natural join t) natural join s**

Q2: **select * from (r natural join u) natural join s**

- Both queries share common subexpression (r natural join s)
- May be useful to compute (r natural join s) once and use it in both queries
 - ▶ But this may be more expensive in some situations
 - e.g. (r natural join s) may be expensive, plans as shown in queries may be cheaper
- **Multiquery optimization**: find best overall plan for a set of queries, exploiting sharing of common subexpressions between queries where it is useful



Multiquery Optimization (Cont.)

- Simple heuristic used in some database systems:
 - optimize each query separately
 - detect and exploit common subexpressions in the individual optimal query plans
 - ▶ May not always give best plan, but is cheap to implement
 - **Shared scans**: widely used special case of multiquery optimization
- Set of materialized views may share common subexpressions
 - As a result, view maintenance plans may share subexpressions
 - Multiquery optimization can be useful in such situations



Optimization of Updates

□ Halloween problem

**update R set $A = 5 * A$
where $A > 10$**

- If index on A is used to find tuples satisfying $A > 10$, and tuples updated immediately, same tuple may be found (and updated) multiple times
- Solution 1: *Always defer updates*
 - ▶ collect the updates (old and new values of tuples) and update relation and indices in second pass
 - ▶ Drawback: extra overhead even if e.g. update is only on R.B, not on attributes in selection condition
- Solution 2: *Defer only if required*
 - ▶ Perform immediate update if update does not affect attributes in where clause, and deferred updates otherwise.



Parametric Query Optimization

- Example

```
select *  
from r natural join s  
where r.a < $1
```

 - value of parameter \$1 not known at compile time
 - ▶ known only at run time
 - different plans may be optimal for different values of \$1
- Solution 1: optimize at run time, each time query is submitted
 - ▶ can be expensive
- Solution 2: **Parametric Query Optimization:**
 - optimizer generates a set of plans, optimal for different values of \$1
 - ▶ Set of optimal plans usually small for 1 to 3 parameters
 - ▶ Key issue: how to do find set of optimal plans efficiently
 - best one from this set is chosen at run time when \$1 is known
- Solution 3: **Query Plan Caching**
 - If optimizer decides that same plan is likely to be optimal for all parameter values, it caches plan and reuses it, else reoptimize each time
 - Implemented in many database systems



Plan Stability Across Optimizer Changes

- What if 95% of plans are faster on database/optimizer version N+1 than on N, but 5% are slower?
 - Why should plans be slower on new improved optimizer?
 - ▶ Answer: Two wrongs can make a right, fixing one wrong can make things worse!
- Approaches:
 - Allow hints for tuning queries
 - ▶ Not practical for migrating large systems with no access to source code
 - Set optimization level, default to version N (Oracle)
 - ▶ And migrate one query at a time after testing both plans on new optimizer
 - Save plan from version N, and give it to optimizer version N+1
 - ▶ Sybase, XML representation of plans (SQL Server)



Adaptive Query Processing

- Some systems support adaptive operators that change execution algorithm on the fly
 - E.g., (indexed) nested loops join or hash join chosen at run time, depending on size of outer input
- Other systems allow monitoring the behavior of plan at run time and adapt plan
 - E.g., if statistics such as number of rows is found to be very different in reality from what optimizer estimated
 - Outer join

$$R \text{ join } S \neq R \text{ anti-join } S \neq S \text{ anti-join } R$$
 - Can stop execution, compute fresh plan, and restart
 - ▶ But must avoid too many such restarts

